Assignment 2

The purpose of this assignment is to compare different edge detection methods, implement connected component detection, and apply your knowledge of edge and circle detection (a form of geometric primitive extraction) to segment and identify the circular boundaries between the pupil–iris and iris–sclera in eye images.

Task 1. Use the OpenCV built-in functions for:

1. The Marr-Hildreth edge detector
2. The Canny edge detector

Apply these edge detectors on "img1.tif". Remember, different parameter values can significantly impact the performance of these edge detectors. Experiment with different range of parameters and choose the ones giving best results.

Store the output images as "img1_marr_hildreth", "img1_canny", respectively.

Compare the results of the two edge detectors. Are there any differences in the results? Why do you think the edge detectors are giving different results, if they are? Which factors contributed more to the differences – the parameters or the algorithms themselves? The comparison and discussion should be done in "Assignment 2_Report.pdf".

Task 2. Implement **from scratch** the algorithm to group adjacent pixels in an edge map, as given in the "Edge Detection" lecture slides. Apply your algorithms on the edge maps generated using Tasks 1.1 and 1.2 and mark different connected regions in different colours.

Store the output images as "img1_connected_marr_hildreth", "img1_connected_canny", respectively.

Compare the results. Which edge map resulted in more connected components (regions)? Why? Which output represents better features? The comparison and discussion should be done in "Assignment 2_Report.pdf".

Task 3. The human eye has distinct regions that can be segmented for various applications, notably in biometrics for iris recognition. The most prominent boundaries are between the pupil and the iris, and between the iris and the sclera (white part of the eye). This task will test your ability to identify and demarcate these regions.

You will be provided with 5 eye images. Each image contains one eye.

For each eye image:

1. Detect the pupil circle (boundary between the pupil and the iris).
2. Detect the iris circle (boundary between the iris and the sclera).
3. Overlay the detected circles on the original images and store these new images.

4. Also, save the corresponding edge images. Partial marks will be awarded if a circle is not precisely detected but a commendable edge map indicating the potential boundary is generated.

Store the output images as “eye1_circles”, “eye1_edges”, “eye2_circles”, “eye2_edges”, , … “eye5_circles”, “eye5_edges, respectively.

Deliverables

Your deliverable should include the following material:

1. **Complete source code.** This should be complete, so that it could be easily executed on another system without additional modifications. Your code **will not run** on another system if you are using any 3rd party library or if you have used **absolute paths** for reading or writing images. The code may contain Python file(s) (.py) or Jupyter Notebook(s) (.ipynb).
2. **Resulting output images**. Make sure that you include all output images. Also, make sure to follow the given names for the output images. Non-compliance will result in marks deduction.
3. **Documentation in pdf format.** Name the file “Assignment 2_Report.pdf”. Make sure to include your output images in the report when discussing results.

Report

- PDF format: Assignment 2_Report.pdf.
- Include:
    - Brief overview of each task.
    - Discussion of results, including visual comparison, wherever applicable.
    - Implementation challenges, if any.
    - Maximum 2 pages of text (excluding images).
    - Ensure the report is **self-contained** so a grader can evaluate your work without running your code.

Instructions

- Store all output images in “tif” format.
- Follow folder structure exactly and keep output filenames exactly as specified.
- Use relative paths for images.
- **Code should run end-to-end without modification.**
- Write clean, readable, and well-structured code and comment code where necessary.
- You must complete **Task 2** by implementing from scratch, i.e., **do not use any built-in OpenCV or any other third-part library functions** except those for reading, writing, displaying, and changing formats of images.
- **For Task 3, your code must work effectively on unseen images**. Use a consistent set of parameters that provides good results across all input images. **Avoid fine-tuning parameters specifically for each individual image.**

- You can also use code from the Internet as long as:
    - You have significantly enhanced the original code.
    - The source is properly cited in your report.
    - Your report clearly describes how the code operates.
- Use Python version > 3.6 and OpenCV version > 3.4
- All code that you submit should be original. Do not scour the Internet for solutions.
- Submit the material to MLS in a **single zip file**. Follow the following folder structure for submission.

Assignment 2

```
|
├── Assignment 2_Report.pdf
|
├── Code
|   ├── …
|   ├── …
|   └── …
|
├── Images
|   ├── …
|   ├── …
|   └── …
|
├── Output Images
|   ├── …
|   ├── …
|   └── …
|
```

Marking Scheme

Task	**Marks**
Task 1. Edge Detectors	8
Task 2. Connected Components	7
Task 3. Iris Segmentation	20
Report	10
Instructions Compliance and Code Quality	6
Total	**51**